EFFECTIVE JAVA

Item 13
Sobrescreva clone com cuidado

Capítulo 3 · Métodos comuns a todos os objetos

O QUE SIGNIFICA COPIAR?

01 Cloneable e o contrato de clone

02 PhoneNumber e herança

03 Stack: array e elementos

04 HashTable: cadeias de nós

05 Cuidados na implementação

06 Construtores e factories de cópia

CLONEABLE NÃO DECLARA CLONE

Mixin

Acrescenta uma capacidade.

Comparable permite comparar.

Cloneable

Zero métodos.

Object.clone continua protected.

Implementar o marcador não publica uma operação de cópia.

UMA INTERFACE COM OPERAÇÃO E UM MARCADOR

contract/CloneableDemo.java

```
public interface Describable {
    String describe();
}
```

contract/CloneableDemo.java

```
public static final class MarkerOnly implements Cloneable {
}
```

contract/CloneableDemoTest.java

```
void mixinExposesOperation() {
    CloneableDemo.Describable value = () -> "Book Club";

    assertThat(value.describe()).isEqualTo("Book Club");
    assertThat(Cloneable.class.getDeclaredMethods()).isEmpty();
}
```

contract/CloneableDemoTest.java

```
void markerDoesNotGrantAccess() throws ReflectiveOperationException {
    var method = Object.class.getDeclaredMethod("clone");
    var value = new CloneableDemo.MarkerOnly();

    assertThatThrownBy(() -> value.getClass().getMethod("clone"))
            .isInstanceOf(NoSuchMethodException.class);
    assertThatThrownBy(() -> method.invoke(value)).isInstanceOf(IllegalAccessException.class);
}
```

TRÊS CONVENÇÕES

x.clone() != x

x.clone().getClass() ==
x.getClass()

x.clone().equals(x)

Outra instância · mesmo tipo · igualdade depende de equals

O TESTE: EQUALS PODE COMPARAR IDENTIDADE

contract/PhoneNumberTest.java

```
void inheritedEqualsUsesIdentity() {
    var original = new Stack();

    assertThat(original.clone()).isNotSameAs(original).isNotEqualTo(original);
}
```

PHONENUMBER: MESMOS VALORES, OUTRA INSTÂNCIA

Object.clone copia os campos, sem executar construtores.

Primitivos: valores copiados.

Referências: os mesmos objetos.

contract/PhoneNumber.java

```java
@Override
public PhoneNumber clone() {
    try {
        return (PhoneNumber) super.clone();
    } catch (CloneNotSupportedException impossible) {
        throw new AssertionError(impossible);
    }
}
```

contract/PhoneNumberTest.java

```
void clonePreservesValue() {
    var original = new PhoneNumber(313, 414, 533);
    int before = PhoneNumber.constructorCalls();

    PhoneNumber copy = original.clone();

    assertThat(copy).isNotSameAs(original).isEqualTo(original);
    assertThat(copy.getClass()).isEqualTo(original.getClass());
    assertThat(copy.hashCode()).isEqualTo(original.hashCode());
    assertThat(PhoneNumber.constructorCalls()).isEqualTo(before);
}
```

INSTANCIAR A CLASSE BASE PERDE O SUBTIPO

new Parent() continua sendo Parent,
mesmo quando o original é Child.

A cadeia de super.clone preserva a classe real.

inheritance/InheritanceExample.java

```
public InheritanceExample brokenClone() {
    return new InheritanceExample();
}
```

inheritance/InheritanceExample.java

```
@Override
protected InheritanceExample clone() throws CloneNotSupportedException {
    return (InheritanceExample) super.clone();
}
```

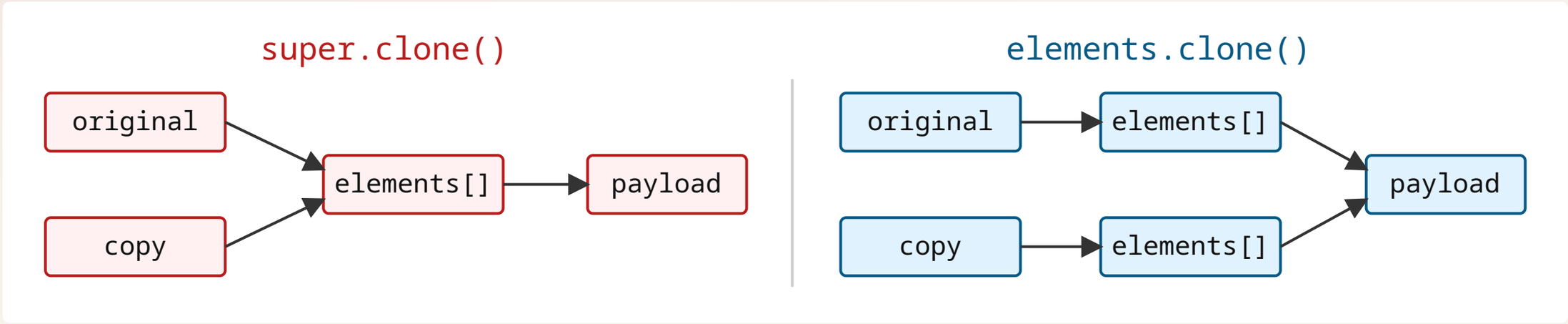
O TESTE: O CLONE CONTINUA SENDO CHILD

inheritance/InheritanceExampleTest.java

```
void superClonePreservesSubtype() {
    var original = new InheritanceExample.Child();

    assertThat(original.brokenClone()).isExactlyInstanceOf(InheritanceExample.class);
    assertThatThrownBy(() -> InheritanceExample.Child.class.cast(original.brokenClone()))
            .isInstanceOf(ClassCastException.class);
    var copy = (InheritanceExample.Child) original.clone();
    assertThat(copy).isNotSameAs(original).isExactlyInstanceOf(original.getClass());
    assertThat(copy.label()).isEqualTo("backend");
}
```

STACK: OUTRA PILHA, O MESMO ARRAY?



Copiar a pilha não separa automaticamente seu armazenamento.

mutable/Stack.java

```
public Stack shallowCopy() {
    try {
        return (Stack) super.clone();
    } catch (CloneNotSupportedException impossible) {
        throw new AssertionError(impossible);
    }
}
```

mutable/Stack.java

```
@Override
public Stack clone() {
    Stack copy = shallowCopy();
    copy.elements = elements.clone();
    return copy;
}
```

O TESTE: POP NÃO PODE CORROMPER O ORIGINAL

mutable/StackTest.java

```
void cloneOwnsBackingArray() {
    var original = new Stack();
    original.push("top");
    original.shallowCopy().pop();
    assertThat(original.pop()).isNull();

    original.push("top");
    original.clone().pop();
    assertThat(original.pop()).isEqualTo("top");
}
```

ARRAY NOVO.
OS MESMOS ELEMENTOS.

StringBuilder[] copia
referências aos builders.
Para isolar seus estados, copie também cada elemento mutável.

O TESTE: COPIAR O ARRAY E COPIAR O ELEMENTO

mutable/StackTest.java

```
void arrayCloneSharesElements() {
    StringBuilder[] original = {new StringBuilder("A")};
    StringBuilder[] copy = original.clone();
    copy[0].append("B");
    assertThat(copy).isNotSameAs(original);
    assertThat(copy[0]).isSameAs(original[0]);
    assertThat(original[0].toString()).isEqualTo("AB");

    StringBuilder[] independent = {new StringBuilder(original[0])};
    independent[0].append("C");
    assertThat(original[0].toString()).isEqualTo("AB");
    assertThat(independent[0].toString()).isEqualTo("ABC");
}
```

HASHTABLE: BUCKETS NOVOS, NÓS COMPARTILHADOS

Copiar o array de buckets não copia os nós das suas cadeias.

A correção precisa reconstruir todos os nós.

mutable/HashTable.java

```
private Entry copyChain() {
    Entry head = new Entry(key, value, null);
    Entry tail = head;
    for (Entry node = next; node != null; node = node.next) {
        tail.next = new Entry(node.key, node.value, null);
        tail = tail.next;
    }
    return head;
}
```

mutable/HashTable.java

```
@Override
public HashTable clone() {
    HashTable copy = shallowBucketsCopy();
    for (int i = 0; i < buckets.length; i++) {
        copy.buckets[i] = buckets[i] == null ? null : buckets[i].copyChain();
    }
    return copy;
}
```

mutable/HashTableTest.java

```java
void cloneOwnsChain() {
    var original = new HashTable();
    original.put("Aa", "first");
    original.put("BB", "second");
    assertThat("Aa".hashCode()).isEqualTo("BB".hashCode());
    original.shallowBucketsCopy().put("Aa", "shared");
    assertThat(original.get("Aa")).isEqualTo("shared");

    var copy = original.clone();
    copy.put("Aa", "copy tail");
    copy.put("BB", "copy head");
    assertThat(original.get("Aa")).isEqualTo("shared");
    assertThat(original.get("BB")).isEqualTo("second");
    assertThat(copy.get("Aa")).isEqualTo("copy tail");
    assertThat(copy.get("BB")).isEqualTo("copy head");
}
```

PRESERVE AS INVARIANTES

Referências final

final não torna a lista imutável.
clone não pode substituir esse campo.

Exceção verificada

Sem Cloneable, Object.clone lança CloneNotSupportedException.

contract/CloneableDemoTest.java

```
void finalRepairDoesNotCompile() throws IOException {
    assertThat(compile("FinalRepair")).contains("compiler.err.cant.assign.val.to.var");
}
```

contract/CloneableDemoTest.java

```
void missingMarkerRejectsClone() {
    assertThatThrownBy(CloneableDemo::unsupportedCopy)
            .isInstanceOf(CloneNotSupportedException.class);
}
```

CONSTRUA A CÓPIA EXPLICITAMENTE

Copy constructor

new Account(original)

Copy factory

Account.copyOf(original)

Construção normal, campos final e política de cópia definida pela classe.

alternatives/Account.java

```
private final List<String> labels;

public Account(List<String> labels) {
    this.labels = new ArrayList<>(labels);
}

public Account(Account source) {
    this(source.labels);
}

public static Account copyOf(Account source) {
    return new Account(source);
}
```

O TESTE: CADA CÓPIA POSSUI SUA PRÓPRIA LISTA

alternatives/AccountTest.java

```
void copiesOwnFinalList() {
    var original = new Account(List.of("primary"));
    var constructorCopy = new Account(original);
    var factoryCopy = Account.copyOf(original);
    constructorCopy.addLabel("constructor");
    factoryCopy.addLabel("factory");

    assertThat(original.labels()).containsExactly("primary");
    assertThat(constructorCopy.labels()).containsExactly("primary", "constructor");
    assertThat(factoryCopy.labels()).containsExactly("primary", "factory");
    assertThat(factoryCopy).isNotSameAs(original);
}
```

ITEM 13

- ✓ Cloneable não oferece uma operação pública
- ✓ Object.clone faz uma cópia superficial
- ✓ Copie o estado mutável que precisa de independência
- ✓ Preserve as invariantes e a classe concreta
- ✓ Prefira construtores e factories em novos modelos

DEFINA A CÓPIA.

O que deve ser preservado?

O que precisa ser independente?